gemini-conversation

Exported on: 10/05/2026, 12:45:11 PM

User

I am working on a Pygame arm wrestling game. Here is my current game_engine.py code. There is a bug where pressing the Left and Right arrow keys increases self.arm_position, pushing toward the computer. Please modify the handle_event method so that alternating these keys decreases the position toward the player's winning threshold (-100.0)

Gemini

Change the `+=` operators to `-=` in the `handle_event` method within `game_engine.py` so the arm position moves toward the negative threshold when the player alternates the arrow keys.

PY

Python

@ y0

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if self.stamina <= 10:
            return

        if event.key == pygame.K_LEFT:
            if self.last_key != pygame.K_LEFT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_LEFT
        elif event.key == pygame.K_RIGHT:
            if self.last_key != pygame.K_RIGHT:
                self.arm_position -= 4.2
                self.stamina = max(0.0, self.stamina - 2.0)
                self.last_key = pygame.K_RIGHT
```

User

Now, update the update method to introduce a dynamic AI surge cycle. The AI should periodically enter a brief high-power surge with increased pushing force, followed by a cooldown period with reduced resistance. Keep the gameplay balanced.

Gemini

Replace the existing `update` method in `game_engine.py` to use `pygame.time.get_ticks()` for managing the AI's phase. This implements a 6-second cycle that rotates through normal resistance, a high-power surge, and a vulnerable cooldown.

PY

Python

@ y0

```
    def update(self):
        if self.game_state != "PLAYING":
            return

        # 6-second dynamic AI cycle using milliseconds
        cycle_time = pygame.time.get_ticks() % 6000

        if cycle_time < 3500:
            phase_multiplier = 1.0 # Normal pushing phase
        elif cycle_time < 4500:
            phase_multiplier = 2.2 # 1-second high-power surge
        else:
            phase_multiplier = 0.4 # 1.5-second cooldown/fatigue

        ai_variance = random.uniform(0.3, 1.0)
        self.arm_position += (self.ai_strength * ai_variance) * phase_multiplier

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.8)

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"
        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"
```

This configuration keeps the match balanced by forcing the player to manage their stamina during the 1-second surge (2.2x strength) and capitalize on the subsequent 1.5-second cooldown (0.4x strength) to push back toward the negative win threshold. Since `get_ticks()` runs continuously, the AI phase naturally desynchronizes from round restarts, adding slight unpredictability to each new match.

User

Next, build on the surge cycle by adding visual cues in the render method. Display an active warning indicator on the screen when the AI enters its power surge. Also, display an exhaustion state warning whenever the player's stamina drops below 10.

Gemini

Insert the visual warning text just below the respective player and computer headers in the `render` method. This recalculates the `get_ticks()` cycle during rendering to sync the AI's surge text with its background logic, and checks the existing 10-stamina threshold for the player.

PY

Python

@ y0

```
    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240, 240
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
        computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        # --- Visual Cues ---
        if self.game_state == "PLAYING":
            if self.stamina <= 10:
                exhaustion_warn = self.font_med.render("EXHAUSTED!", True, (240, 60, 6
                screen.blit(exhaustion_warn, (60, 80))

            cycle_time = pygame.time.get_ticks() % 6000
            if 3500 <= cycle_time < 4500:
                surge_warn = self.font_med.render("SURGE!", True, (240, 60, 60))
                screen.blit(surge_warn, (self.width - 150, 80))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
        pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

        pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width // 2

        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235
```

```
            p_shoulder = (70, 330)
            p_elbow = (140, 215)
            c_shoulder = (self.width - 70, 330)
            c_elbow = (self.width - 140, 215)

            pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
            pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
            pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

            pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
            pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
            pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

            pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
            pygame.draw.circle(screen, (160, 115, 85), (hand x, hand y), 24, width=3)

            stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
            screen.blit(stamina_label, (40, 445))

            stamina_bg = pygame.Rect(140, 448, 240, 22)
            stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina / self.max_stamin
            pygame.draw.rect(screen, (45, 50, 60), stamina_bg, border_radius=6)
            bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)
            pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

            if self.game_state == "GAME_OVER":
                overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
                overlay.fill((0, 0, 0, 200))
                screen.blit(overlay, (0, 0))

                win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else "COMPU
                color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
                text_surf = self.font_big.render(win_text, True, color)
                screen.blit(
                    text_surf,
                    (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
                )

                restart_surf = self.font_med.render(
                    "Press [R] to Rematch", True, (240, 240, 240)
                )
                screen.blit(
                    restart_surf,
                    (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 1
                )
```

User

Finally, reward the player for strategic timing. Modify the logic so that if the player pushes the right arrow key exactly as the AI's surge ends and its cooldown begins, they are granted a temporary stamina recovery boost and double push strength. Provide the complete, finalized game_engine.py code.

Gemini

Here is the complete, finalized `game_engine.py` with the new `boost_end_time` logic integrated. A 150-millisecond window is provided right as the cycle crosses the 4500ms mark to detect the perfectly timed right arrow press, which triggers a 1.5-second boost to both push strength and stamina recovery.

PY

Python

@ y0

```
import math
import random
import pygame

class GameEngine:

    def _init_(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

        self.boost_end_time = 0

    def handle_event(self, event):
        if self.game state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return
```

```
            if event.type == pygame.KEYDOWN:
                if self.stamina <= 10:
                    return

                current_ticks = pygame.time.get_ticks()
                push_strength = 8.4 if current_ticks < self.boost_end_time else 4.2

                if event.key == pygame.K_LEFT:
                    if self.last_key != pygame.K_LEFT:
                        self.arm_position -= push_strength
                        self.stamina = max(0.0, self.stamina - 2.0)
                        self.last_key = pygame.K_LEFT
                elif event.key == pygame.K_RIGHT:
                    if self.last_key != pygame.K_RIGHT:
                        # Check for perfect timing: exactly as AI surge ends (4500ms)
                        cycle_time = current_ticks % 6000
                        if 4500 <= cycle_time <= 4650:
                            # Grant a 1.5 second momentum boost
                            self.boost_end_time = current_ticks + 1500
                            push_strength = 8.4 # Apply double strength immediately

                        self.arm_position -= push_strength
                        self.stamina = max(0.0, self.stamina - 2.0)
                        self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        current_ticks = pygame.time.get_ticks()
        cycle_time = current_ticks % 6000

        if cycle_time < 3500:
            phase_multiplier = 1.0 # Normal pushing phase
        elif cycle_time < 4500:
            phase_multiplier = 2.2 # 1-second high-power surge
        else:
            phase_multiplier = 0.4 # 1.5-second cooldown/fatigue

        ai_variance = random.uniform(0.3, 1.0)
        self.arm_position += (self.ai_strength * ai_variance) * phase_multiplier

        # Apply temporary stamina recovery boost if active
        recovery_rate = 3.0 if current_ticks < self.boost_end_time else 0.8
        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + recovery_rate)
```

```
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"
        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.boost_end_time = 0

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240, 240
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
        computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        # --- Visual Cues ---
        if self.game_state == "PLAYING":
            if self.stamina <= 10:
                exhaustion_warn = self.font_med.render("EXHAUSTED!", True, (240, 60, 6
                screen.blit(exhaustion_warn, (60, 80))

            current_ticks = pygame.time.get_ticks()
            cycle_time = current_ticks % 6000

            if 3500 <= cycle_time < 4500:
                surge_warn = self.font_med.render("SURGE!", True, (240, 60, 60))
                screen.blit(surge_warn, (self.width - 150, 80))

            if current_ticks < self.boost_end_time:
                boost_msg = self.font_med.render("PERFECT PARRY!", True, (100, 200, 25
                screen.blit(boost_msg, (60, 105))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
```

```
            pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

            pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width // 2

            offset_x = (self.arm_position / self.target_limit) * 95
            hand_x = (self.width // 2) + int(offset_x)
            hand_y = 235

            p_shoulder = (70, 330)
            p_elbow = (140, 215)
            c_shoulder = (self.width - 70, 330)
            c_elbow = (self.width - 140, 215)

            pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
            pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
            pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

            pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
            pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
            pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

            pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
            pygame.draw.circle(screen, (160, 115, 85), (hand x, hand y), 24, width=3)

            stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
            screen.blit(stamina_label, (40, 445))

            stamina_bg = pygame.Rect(140, 448, 240, 22)
            stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina / self.max_stamin
            pygame.draw.rect(screen, (45, 50, 60), stamina_bg, border_radius=6)

            # Turn stamina bar blue to visually reflect the boost state
            if current_ticks < getattr(self, 'boost_end_time', 0):
                bar_color = (100, 200, 255)
            else:
                bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)

            pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

            if self.game_state == "GAME_OVER":
                overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
                overlay.fill((0, 0, 0, 200))
                screen.blit(overlay, (0, 0))

                win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else "COMPU
                color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
```

```
            text_surf = self.font_big.render(win_text, True, color)
            screen.blit(
                text_surf,
                (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 1
            )
```